

SIMATS ENGINEERING

ASSESSMENT TOOL 2

**COURSE CODE: CSA14
DESIGN**

COURSE NAME: COMPILER

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:100

Annexure A

Debugging

Code of Conduct:

*I Arunprasath R (192524077)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
Arunprasath R

Q. No	Understanding of Debugging Concepts (25)	Error Identification(25)	Root Causes Analysis (20)	Error Corrections and Solution Design(20)	Testing and Validation(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

Debugging

Q1 Consider the following syntax-directed definition:

$$E \rightarrow E1 + T \quad \{ E.val = T.val \}$$
$$E \rightarrow T \quad \{ E.val = T.val \}$$
$$T \rightarrow \text{num} \quad \{ T.val = \text{num.lexval} \}$$

For the input:

3 + 5

the compiler produces:

$E.val = 5$

Debug the given syntax-directed definition. Identify the incorrect semantic rule, explain its effect on attribute evaluation, provide the corrected rule, and determine the correct value of the expression.

Ans:

1.1 Identification of the Error

The incorrect semantic rule is:

$$E \rightarrow E1 + T \quad \{ E.val = T.val \}$$

The production represents an addition operation:

$$E \rightarrow E1 + T$$

Therefore, the value of E must contain the values of **both operands**.

However, the given semantic rule considers only T.val and completely ignores E1.val.

1.2 Attribute Evaluation

For the input:

3 + 5

the relevant structure is:

$$\begin{array}{c}
 E \\
 / \mid \backslash \\
 E1 \ + \ T \\
 | \quad | \\
 T \quad \text{num} \\
 | \quad | \\
 \text{num} \quad 5 \\
 | \\
 3
 \end{array}$$

From:

$$T \rightarrow \text{num}$$

we obtain:

$$E1.\text{val} = 3$$

$$T.\text{val} = 5$$

Using the incorrect rule:

$$E.\text{val} = T.\text{val}$$

$$= 5$$

Therefore, the compiler incorrectly produces:

$$E.\text{val} = 5$$

1.3 Correct Semantic Rule

The correct rule is:

$$E \rightarrow E1 + T \quad \{ E.\text{val} = E1.\text{val} + T.\text{val} \}$$

The remaining rules are:

$$E \rightarrow T \quad \{ E.\text{val} = T.\text{val} \}$$

$T \rightarrow \text{num} \quad \{ T.\text{val} = \text{num.lexval} \}$

1.4 Correct Evaluation

For:

$3 + 5$

we have:

$E1.\text{val} = 3$

$T.\text{val} = 5$

Therefore:

$E.\text{val} = E1.\text{val} + T.\text{val}$

$= 3 + 5$

$= 8$

Result

Error: The semantic rule ignores $E1.\text{val}$.

Correct rule:

$E \rightarrow E1 + T \quad \{ E.\text{val} = E1.\text{val} + T.\text{val} \}$

Correct expression value:

$E.\text{val} = 8$

```

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>notepad q1.c

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>dir q1.c
Volume in drive C is OS
Volume Serial Number is BC6A-BF46

Directory of C:\Users\arunp\OneDrive\Documents\Compiler - LAB

13-08-2026  10:56                922 q1.c
             1 File(s)                922 bytes
             0 Dir(s)  326,501,445,632 bytes free

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>q1.c.txt
'q1.c.txt' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>q1.c

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc q1.c -o q1.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>q1.exe
Syntax-Directed Definition Debugging
-----
Enter first number: 3
Enter second number: 5

Incorrect Semantic Rule:
E.val = T.val
E.val = 5

Correct Semantic Rule:
E.val = E1.val + T.val
E.val = 3 + 5
E.val = 8

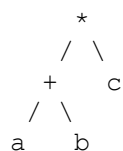
Final Correct Value = 8

```

Q2 . For the expression:

$a + b * c$

the compiler constructs the following syntax tree:



The compiler subsequently generates:

```

t1 = a + b
t2 = t1 * c

```

Debug the syntax-tree construction. Analyze the relationship between operator precedence and the generated tree, construct the corrected syntax tree, and generate the correct intermediate representation.

Ans:

2.1 Operator Precedence Analysis

The expression contains two operators:

+

*

Multiplication has higher precedence than addition.

Therefore:

$a + b * c$

must be interpreted as:

$a + (b * c)$

It must not be interpreted as:

$(a + b) * c$

2.2 Error in the Given Syntax Tree

The given tree is:

```
      *
     /\
    +  c
   /\
  a  b
```

This tree represents:

$(a + b) * c$

Therefore, the tree violates the required operator precedence.

2.3 Correct Syntax Tree

The corrected tree is:

$$\begin{array}{c} + \\ / \backslash \\ a \quad * \\ / \backslash \\ b \quad c \end{array}$$

This represents:

$$a + (b * c)$$

2.4 Error in Intermediate Representation

The compiler generated:

$$t1 = a + b$$
$$t2 = t1 * c$$

This corresponds to:

$$(a + b) * c$$

Therefore, the intermediate representation is also incorrect.

2.5 Correct Intermediate Representation

Since multiplication must be performed first:

$$t1 = b * c$$
$$t2 = a + t1$$

This correctly represents:

$$a + (b * c)$$

Result

Error: The syntax tree incorrectly gives higher priority to +.

Correct syntax tree:

$$+$$

/\

a *

/\

b c

Correct three-address code:

t1 = b * c

t2 = a + t1

```
C:\Users\arunp\OneDrive\Documents\Compiler - LAB>dir q2.c
Volume in drive C is OS
Volume Serial Number is BC6A-BF46

Directory of C:\Users\arunp\OneDrive\Documents\Compiler - LAB

13-08-2026  11:06                964 q2.c
              1 File(s)              964 bytes
              0 Dir(s)  326,230,757,376 bytes free

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc q2.c -o q2.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>q2.exe
Syntax Tree Debugging
-----
Enter expression (example: a+b*c): a+b*c

Given Expression: a + b * c

Incorrect Syntax Tree:
      *
     /\
    +  c
   /\
  a  b

This represents: (a + b) * c

Correct Syntax Tree:
      +
     /\
    a  *
       /\
      b  c

This represents: a + (b * c)

Correct Three Address Code:
t1 = b * c
t2 = a + t1

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>
```

Q3. For the expression:

$3 + 4 * 5$

using:

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow \text{num}$

the compiler annotates the root with:

$E.\text{val} = 35$

Ans:

3.1 Grammar Analysis

The grammar separates addition and multiplication:

$E \rightarrow E + T$

$T \rightarrow T * F$

This structure gives multiplication higher precedence than addition.

Therefore:

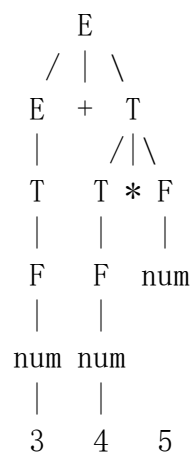
$3 + 4 * 5$

is interpreted as:

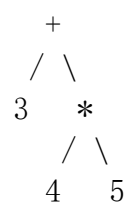
$3 + (4 * 5)$

3.2 Parse Structure

The relevant structure is:



Conceptually, the corresponding expression tree is:



3.3 Synthesized Attribute Evaluation

From:

$F \rightarrow \text{num}$

the leaf values are:

$F.val = 3$

$F.val = 4$

$F.val = 5$

For:

$T \rightarrow T * F$

we obtain:

$T.val = 4 * 5$

$= 20$

Then, for:

$E \rightarrow E + T$

we obtain:

$E.val = 3 + 20$

$= 23$

3.4 Identification of the Error

The compiler reports:

$E.val = 35$

However:

$3 + 4 * 5$

$= 3 + 20$

$= 23$

The value 35 corresponds to:

$(3 + 4) * 5$

$= 7 * 5$

$= 35$

This violates the operator precedence represented by the grammar.

Result

The reported value:

$E.val = 35$

is incorrect.

The correct evaluation is:

$4 * 5 = 20$

$3 + 20 = 23$

Therefore:

$E.val = 23$

```

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>dir q3.c
Volume in drive C is OS
Volume Serial Number is BC6A-BF46

Directory of C:\Users\arunp\OneDrive\Documents\Compiler - LAB

13-08-2026  11:09                1,477 q3.c
              1 File(s)                1,477 bytes
              0 Dir(s)  326,251,622,400 bytes free

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc q3.c -o q3.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>q3.exe
Annotated Parse Tree Debugging
-----
Expression: 3 + 4 * 5

Step 1: Leaf Attribute Values
F1.val = 3
F2.val = 4
F3.val = 5

Step 2: Multiplication Attribute
T.val = F2.val * F3.val
T.val = 4 * 5
T.val = 20

Step 3: Addition Attribute
E.val = E1.val + T.val
E.val = 3 + 20
E.val = 23

Incorrect Compiler Result:
E.val = (3 + 4) * 5
E.val = 35

Correct Evaluation:
3 + (4 * 5)
3 + 20
E.val = 23

Final Correct Value = 23

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>

```

Analyze the annotated parse tree. Trace the synthesized attributes from the leaves to the root, identify the incorrect computation, and determine the correct value of the expression.

Q4 . The symbol table contains:

Identifier	Type
A	int
B	float
C	int
X	float

For the statement:

```
x = a + b * c;
```

the compiler reports:

```
Type Error: incompatible operands
```

Debug the type-checking process. Analyze each subexpression, determine the type of $b * c$ and $a + (b * c)$, identify where type conversion is required, and determine whether the assignment to x is valid.

Ans:

4.1 Operator Precedence

The expression is:

$$a + b * c$$

Since multiplication has higher precedence:

$$a + (b * c)$$

must be evaluated.

4.2 Type of $b * c$

The operands are:

$b : \text{float}$

$c : \text{int}$

The types are different.

Under standard numeric conversion rules, the int operand can be converted to float.

Therefore:

$$c : \text{int} \rightarrow \text{float}$$

The multiplication becomes conceptually:

$$b * \text{float}(c)$$

and its result is:

float

Therefore:

$b * c : \text{float}$

4.3 Type of $a + (b * c)$

We have:

$a : \text{int}$

$b * c : \text{float}$

The integer operand must be converted to float:

$a : \text{int} \rightarrow \text{float}$

Therefore:

$\text{float}(a) + (b * c)$

has type:

float

Hence:

$a + b * c : \text{float}$

4.4 Assignment Analysis

The assignment is:

$x = a + b * c$

where:

$x : \text{float}$

$a + b * c : \text{float}$

Therefore, both sides have compatible types.

The assignment is valid.

4.5 Required Conversions

The conceptual conversion sequence is:

$c \rightarrow \text{float}$

$b * \text{float}(c)$

followed by:

$a \rightarrow \text{float}$

$\text{float}(a) + \text{result}$

A possible three-address representation is:

$t1 = \text{inttofloat}(c)$

$t2 = b * t1$

$t3 = \text{inttofloat}(a)$

$t4 = t3 + t2$

$x = t4$

Result

Expression	Type
B	float
C	int
$b * c$	float
A	int
$a + (b * c)$	float
X	float

Therefore:

$x = a + b * c;$

is **valid after the required numeric conversions**.

The reported error:

Type Error: incompatible operands

is incorrect if the language supports standard $\text{int} \rightarrow \text{float}$ promotion.

```
C:\Windows\System32\cmd.e  X  +  v

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>dir q4.c
Volume in drive C is OS
Volume Serial Number is BC6A-BF46

Directory of C:\Users\arunp\OneDrive\Documents\Compiler - LAB

13-08-2026  11:10                1,306 q4.c
              1 File(s)                1,306 bytes
              0 Dir(s)  326,140,194,816 bytes free

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc q4.c -o q4.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>q4.exe
Type Checking and Type Conversion
-----
Symbol Table:
a : int
b : float
c : int
x : float

Expression: x = a + b * c

Step 1: Checking b * c
b : float
c : int
Converting c from int to float...
b * c : float

Step 2: Checking a + (b * c)
a : int
b * c : float
Converting a from int to float...
a + (b * c) : float

Step 3: Checking assignment
x : float
Right-hand side : float
Types are compatible.

Three Address Code:
t1 = inttofloat(c)
t2 = b * t1
t3 = inttofloat(a)
t4 = t3 + t2
x = t4

Compiler Result:
Valid - No Type Error
```

Q5. Consider:

```
int a;
float b;
int x;
```

and the statements:

```
x = a + b;  
b = a + x;
```

The type checker reports both statements as:

Valid

and does not generate any conversion.

Analyze the type-checking results. Determine the type of each expression, identify the required conversions, and explain how the type checker should process each assignment.

Ans;

5.1 Statement 1: $x = a + b$

The operands are:

$a : \text{int}$

$b : \text{float}$

Therefore:

$a + b$

contains:

$\text{int} + \text{float}$

The integer operand must be promoted to float:

$a : \text{int} \rightarrow \text{float}$

Therefore:

$a + b : \text{float}$

Assignment Check

The assignment is:

$x = a + b$

where:

$x : \text{int}$

$a + b : \text{float}$

Therefore, the compiler must convert:

$\text{float} \rightarrow \text{int}$

This is a **narrowing conversion** and may cause loss of precision.

For example:

$a = 5$

$b = 2.5$

gives:

$a + b = 7.5$

Assigning 7.5 to an integer can result in loss of the fractional portion.

Therefore, if implicit narrowing conversions are not allowed, the statement should be rejected.

A possible explicit conversion is:

$x = \text{int}(a + b);$

5.2 Statement 2: $b = a + x$

The operands are:

$a : \text{int}$

$x : \text{int}$

Therefore:

$a + x : \text{int}$

No conversion is required during the addition.

The assignment is:

$b = a + x$

where:

$b : \text{float}$

$a + x : \text{int}$

The integer result can be promoted to float:

$\text{int} \rightarrow \text{float}$

This is a widening conversion and is normally permitted.

Therefore, the statement is valid.

5.3 Correct Type-Checking Summary

Statement	Expression Type	LHS Type	Required Conversion	Result
$x = a + b$	float	int	$\text{float} \rightarrow \text{int}$	Invalid without explicit narrowing conversion
$b = a + x$	int	float	$\text{int} \rightarrow \text{float}$	Valid

5.4 Possible Intermediate Representation

For:

$b = a + x;$

the compiler can generate:

$t1 = a + x$

$t2 = \text{inttofloat}(t1)$

$b = t2$

For:

$x = a + b;$

the expression requires:

$t1 = \text{inttofloat}(a)$

$t2 = t1 + b$

The resulting float must then be converted to int if the language explicitly permits narrowing:

$t3 = \text{floattoint}(t2)$

$x = t3$

Otherwise, the compiler should report a type error.

Result

The original type-checking result is incomplete because it performs no required conversions.

The correct analysis is:

$x = a + b$

requires:

$\text{int} \rightarrow \text{float}$

during expression evaluation and potentially:

$\text{float} \rightarrow \text{int}$

during assignment.

Therefore, it should not automatically be accepted if implicit narrowing conversion is prohibited.

For:

$b = a + x$

the result is valid because:

$\text{int} \rightarrow \text{float}$

is a widening conversion.

```

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>dir q5.c
Volume in drive C is OS
Volume Serial Number is BC6A-BF46

Directory of C:\Users\arunp\OneDrive\Documents\Compiler - LAB

13-08-2026  11:12                2,596 q5.c
              1 File(s)              2,596 bytes
              0 Dir(s)  326,255,120,384 bytes free

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>q5.c

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc q5.c -o q5.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>q5.exe
Type Checking and Conversion Debugging
-----
Symbol Table:
a : int
b : float
x : int

=====
Statement 1: x = a + b
=====

a : int
b : float

Step 1: Checking a + b
int + float
Converting a from int to float...
a + b : float

Step 2: Checking assignment to x
x : int
Right-hand side : float
Required conversion: float -> int
This is a narrowing conversion.
Implicit narrowing conversion is not allowed.

Result: INVALID without explicit conversion

```

```

Possible explicit conversion:
x = int(a + b)

=====
Statement 2: b = a + x
=====

a : int
x : int

Step 1: Checking a + x
int + int
a + x : int

Step 2: Checking assignment to b
b : float
Right-hand side : int
Required conversion: int -> float
This is a widening conversion.
Implicit widening conversion is allowed.

Result: VALID

=====
Three Address Code
=====

For x = a + b:
t1 = inttofloat(a)
t2 = t1 + b
t3 = floattoint(t2)
x = t3

For b = a + x:
t4 = a + x
t5 = inttofloat(t4)
b = t5

=====
Final Type Checking Results
=====
x = a + b -> INVALID without explicit narrowing conversion
b = a + x -> VALID with int-to-float widening conversion

```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Understanding of Debugging Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Error Identification	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Root Causes Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Error Correction & Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Testing and Validation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand